Natural Computing SoSe24

Exercise Sheet 10 (Mock Exam) – July 16th, 2024

Please mind the following:

- Fill in your **personal information** in the fields below.
- You may use an unmarked dictionary during the exam. **No** additional tools (like calculators, e.g.) might be used.
- Fill in all your answers **directly into this exam sheet**. You may use the backside of the individual sheets or ask for additional paper. In any case, make sure to mark **clearly** which question you are answering. Do not use pens with green or red color or pencils for writing your answers. You may give your answers in both **German and English**.
- Points annotated for the individual tasks only serve as a preliminary guideline.
- At the end of the exam hand in your **whole exam sheet**. Please make sure you do not undo the binder clip!
- To forfeit your exam ("entwerten"), please cross out clearly this **cover page** as well as **all pages** of the exam sheet. This way the exam will not be evaluated and will not be counted as an exam attempt.
- Please place your LMU-Card or student ID as well as photo identification ("Lichtbildausweis") clearly visible on the table next to you. Note that we need to check your data with the data you enter on this cover sheet.

Time Limit: 90 minutes

First Name:	
Last Name:	
Matriculation Number:	

Topic 1	max. 13 pts.	pts.
Topic 2	max. 15 pts.	pts.
Topic 3	max. 12 pts.	pts.
Topic 4	max. 17 pts.	pts.
Topic 5	max. 16 pts.	pts.
Topic 6	max. 17 pts.	pts.
Total max. 90 pts.		pts.

1 Fill in the Blanks **13pts**

(i) Natural computing draws inspiration from the field of _____, for example.

(ii) In mathematician John _____'s Game of Life, we in general use the expression _____ for a pattern with no predecessor.

(iii) The Game of Life is considered _____ because any computer program can be encoded into it.

(iv) The theorem stating that all optimization algorithms perform equally when averaged over all possible target functions is called the _____.

(v) Measures to increase diversity in an evolutionary algorithm's population are used to avoid premature _____.

(vi) The resulting state of a Game of Life board is called _____ after no new patterns are emerging anymore.

(vii) A quantum bit (qubit) can exist in a state called _____ before measurement.

(viii) Two qubits in superposition are _____ so that they can only assume the combined states $(0, 0)$ and $(1, 1)$. One qubit is repeatedly measured to be state 0 in 25% of experiments and 1 in 75% of the experiments. We then measure the second qubit (always after measuring the first one) to be in in the state 1 in _____% of the experiments.

(ix) In genetic algorithms, the step of choosing individuals based on their fitness scores is known as _____.

(x) The crossover operation in genetic algorithms combines the genetic information of _____ to produce offspring.

(xi) The mutation operation in genetic algorithms introduces _____ to individual solutions.

2 Game of Life **15pts**

The following definition was given in the lecture.

Definition 1 (Conway's game of life (standard))**.** Let $G = (V, E)$ be a grid graph with vertices V and (undirected) edges $E \subseteq V \times V$ with a fixed degree of 8 for all nodes. We define $\mathit{surroundings} : V \to \wp(V)$ via

$$\mathit{surroundings}(v) = \{w \mid (v, w) \in E\}$$

so that $|\mathit{surroundings}(v)| = 8$ and $v \notin \mathit{surroundings}(v)$ for all $v \in V$. A state $x \in \mathcal{X}$ is a mapping of vertices to the labels $\{\mathit{dead}, \mathit{alive}\}$, i.e., the state space $\mathcal{X}$ is given via $\mathcal{X} = (V \to \{\mathit{dead}, \mathit{alive}\})$. Let x_t be a state that exists at time step $t \in \mathbb{N}$. We define

$$|v|_{x_t} = |\{w \mid w \in \mathit{surroundings}(v) \wedge x_t(w) = \mathit{alive}\}|.$$

In the game of life, the evolution of a state x_t to its subsequent state x_{t+1} is given deterministically via

$$x_{t+1}(v) = \begin{cases} \mathit{dead} & \text{if } |v|_{x_t} \leq 1, \\ x_t(v) & \text{if } |v|_{x_t} = 2, \\ \mathit{alive} & \text{if } |v|_{x_t} = 3, \\ \mathit{dead} & \text{if } |v|_{x_t} \geq 4, \end{cases}$$

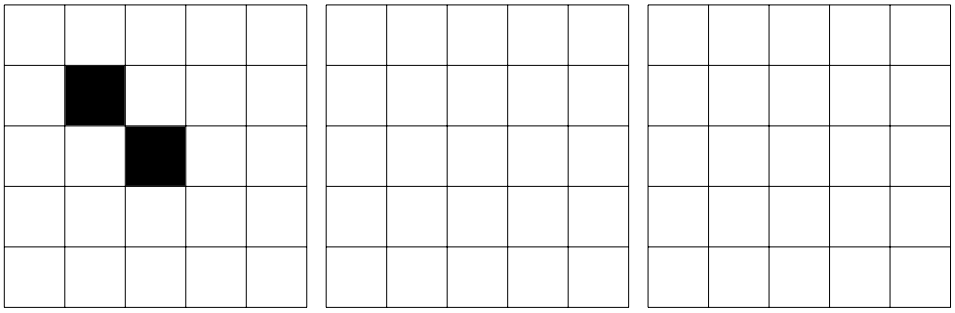
for all $v \in V$. A tuple (G, x_0) is called an instance of the game of life for initial state $x_0 \in \mathcal{X}$.

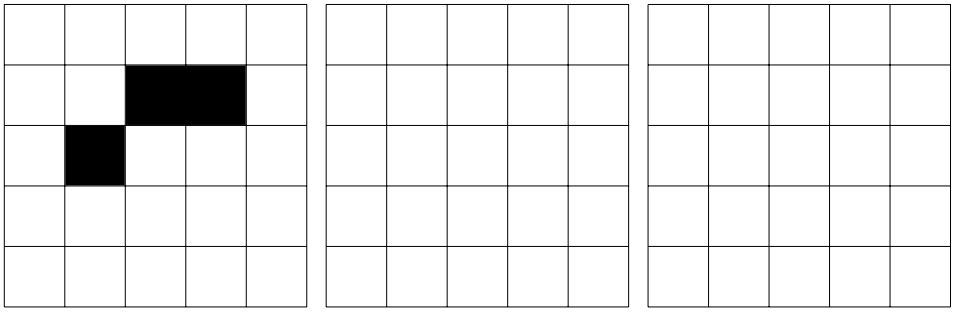
We now introduce a new formal definition:

Definition 2 (exam spaceship)**.** Let (G, x_S) be an instance of the game of life with $G = (V, E)$ played on an $m \times n$ grid. Let $x_{S+1}, x_{S+2}, ...$ be the subsequent states of that instance. (G, x_S) is called an *exam spaceship* if there exists a time frame $\Delta t \in \mathbb{N} \setminus \{0\}$ and there exist displacement values $\Delta x, \Delta y \in \mathbb{Z}$ so that for all $v_{i,j} \in V$ it holds that

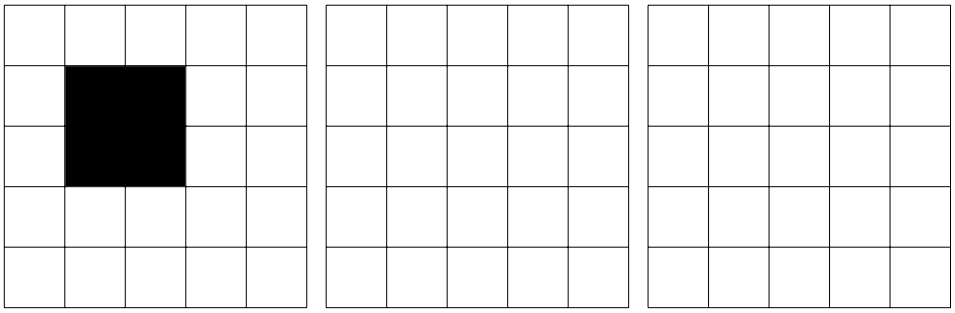
$$x_{S+\Delta t}\left(v_{(i+\Delta y \mod n),\,(j+\Delta x \mod m)}\right) = x_S(v_{i,j}).$$

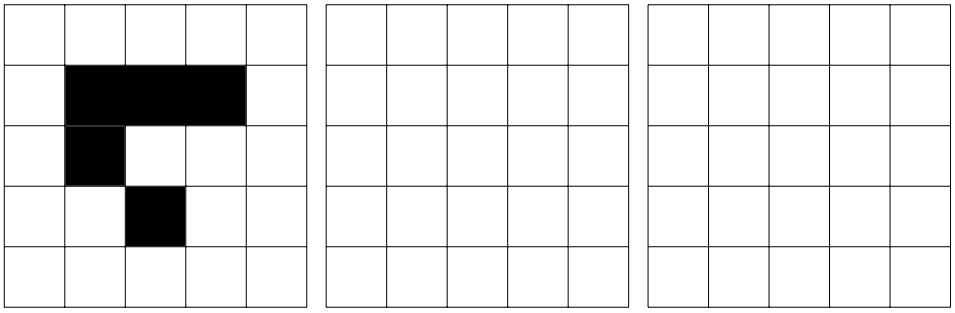
(a) **Show why these two non-empty instances of the game of life are <u>not</u> exam spaceships.** You may use the empty grids to the right of the instances to draw the subsequent states of these instances. (5pts)

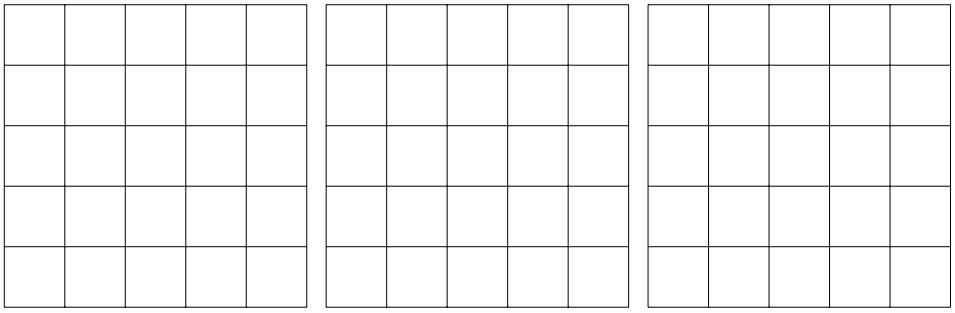




(b) **Show why these two non-empty instances of the game of life are *indeed* exam spaceships. Also give the time frame Δt and the respective displacement values $\Delta x, \Delta y$ for which they fulfill Definition 2.** You may use the empty grids to the right and bottom of the instances to draw the subsequent states of these instances. (7pts)







(c) The definition of an exam spaceship does not match the common definition of a spaceship. Commonly, a spaceship does need to travel across the grid and cannot remain in one place. **Give a constraint on Δt, Δx, and/or Δy from Definition 2 to express this property.** (3pts)

3 Quantum Computing **12pts**

The following definitions were given in the lecture.

Definition 3 (quantum state)**.** Let $\mathcal{X}$ be an arbitrary state space. Let $\mathcal{Q}(\mathcal{X})$ be the space of superposition states with bases in $\mathcal{X}$, i.e., each $q \in \mathcal{Q}(\mathcal{X})$ has the form $q = \sum_{x\in\mathcal{X}} c_x |x\rangle$ where $c_x \in \mathbb{C}$ for any $x \in \mathcal{X}$ is called an amplitude so that $\sum_{x\in\mathcal{X}} |c_x|^2 = 1$ and $|x\rangle$ (read "ket x") denotes a vector corresponding to the state $x \in \mathcal{X}$.

We use the following terminology:

- If for more than one $x \in \mathcal{X}$ it holds that $c_x \neq 0$, then q is called a *superposition* state.
- If $\mathcal{X} = \mathcal{X}_A \times \mathcal{X}_B$, i.e., $\mathcal{X}$ can be constructed as a product of two other state spaces $\mathcal{X}_A, \mathcal{X}_B$, then it follows that q has the form

  $$q = \sum_{\substack{x_A\in\mathcal{X}_A,\\ x_B\in\mathcal{X}_B}} c_{x_A,x_B} |x_A\rangle \otimes |x_B\rangle$$

  where $\otimes$ is the tensor product. If q cannot be written as

  $$q = \big(\sum_{x_A\in\mathcal{X}_A} c_{x_A} |x_A\rangle\big) \otimes \big(\sum_{x_B\in\mathcal{X}_B} c_{x_B} |x_B\rangle\big),$$

  then we call q an *entangled* state.
- If $\mathcal{X} = \{0, 1\}$ (i.e., any $x \in \mathcal{X}$ is a bit), then any $q \in \mathcal{Q}(\mathcal{X})$ is called a *qubit* and can be written as

  $$q = c_0 |0\rangle + c_1 |1\rangle$$

  for $c_0, c_1 \in \mathbb{C}$ with $|c_0|^2 + |c_1|^2 = 1$.
- If $\mathcal{X} = \{0, 1\}^n$ for any $n \in \mathbb{N}$ (i.e., any $x \in \mathcal{X}$ is a bit register), then any $q \in \mathcal{Q}(\mathcal{X})$ is called a *qubit register* of size n and can be written as

  $$q = \sum_{i=0}^{2^n-1} c_i |i\rangle = c_{0\ldots0} |0\ldots0\rangle + \cdots + c_{1\ldots1} |1\ldots1\rangle$$

  where we usually denote i in binary as illustrated above.
- When we *measure* a quantum state $q \in \mathcal{Q}(\mathcal{X})$, we generate classical state $M(q) \in \mathcal{X}$ so that for all $x \in \mathcal{X}$ it holds that

  $$\Pr(M(q) = x) = |c_x|^2.$$

Definition 4 (QUBO)**.** Let $Q \in \mathbb{R}^{n \times n}$ be an arbitrary matrix called QUBO matrix. The process of finding a vector $\vec{x} \in \mathbb{B}^n$ so to

$$\text{minimize } H(\vec{x}) = x^\intercal Q x = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} Q_{ij} x_i x_j$$

is called *quadratic unconstrained binary optimization (QUBO)*. W.l.o.g. we can assume that Q is a (non-strictly) upper triangular matrix.

(a) **Show why** $|a\rangle = 0.5 \cdot |110\rangle + 0.5 \cdot |001\rangle$ **is <u>not a valid</u> quantum state.** (3pts)

(b) **Show why** $|b\rangle = \frac{1}{\sqrt{2}} \cdot |00\rangle + \frac{1}{\sqrt{2}} \cdot |01\rangle$ **is <u>not an entangled</u> quantum state.** (3pts)

(c) Consider the QUBO matrix Q given via $Q = \begin{pmatrix} 3 & 2 & 5 \\ & 4 & 1 \\ & & 7 \end{pmatrix}$.

For the solution candidate $\vec{x} = \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix}$ **compute the energy** $H(\vec{x})$**.** This is not the optimal energy for Q, however. **Give a short argument why the optimal solution candidate for** Q **is** $\vec{x^*} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$ **with** $H(\vec{x^*}) = 0$**.** (6pts)

4 Optimization **17pts**

We give a shortened version of the definition for optimization processes from the lecture and then introduce a new optimization algorithm called *bright side search*.

Definition 5 (optimization (shortened))**.** Let $\mathcal{X}$ be an arbitrary state space. Let $\mathcal{T}$ be an arbitrary set called target space and let $\leq$ be a total order on $\mathcal{T}$. A total function $\tau : \mathcal{X} \to \mathcal{T}$ is called target function. Optimization (minimization/maximization) is the procedure of searching for an $x \in \mathcal{X}$ so that $\tau(x)$ is optimal (minimal/maximal). Unless stated otherwise, we assume minimization.

An optimization run of length $g+1$ is a sequence of states $\langle x_t \rangle_{0 \leq t \leq g}$ with $x_t \in \mathcal{X}$ for all t.

Let $e : \langle \mathcal{X} \rangle \times (\mathcal{X} \to \mathcal{T}) \to \mathcal{X}$ be a possibly randomized or non-deterministic function so that the optimization run $\langle x_t \rangle_{0 \leq t \leq g}$ is produced by calling e repeatedly, i.e., $x_{t+1} = e(\langle x_u \rangle_{0 \leq u \leq t}, \tau)$ for all t, $1 \leq t \leq g$, where x_0 is given externally (e.g., $x_0 =_{def} 42$) or chosen randomly (e.g., $x_0 \sim \mathcal{X}$). An optimization process is a tuple $(\mathcal{X}, \mathcal{T}, \tau, e, \langle x_t \rangle_{0 \leq t \leq g})$.

Algorithm 1 (bright side search)**.** Let $\mathcal{D} = (\mathcal{X}, \mathcal{T}, \tau, e, \langle x_u \rangle_{0 \leq u \leq t})$ be an optimization process. We assume $\mathcal{X} = [0; 100] \subset \mathbb{N}$ and $\mathcal{T} = [0; 100] \subset \mathbb{N}$. The process $\mathcal{D}$ continues via bright side search iff e is given via

$$e(\langle x_u \rangle_{0 \leq u \leq t}, \tau) = x_{t+1} = \begin{cases} \lceil x_t/2 \rceil & \text{if } \tau(x_t - 1) < \tau(x_t + 1), \\ \lfloor x_t + (100 - x_t)/2 \rfloor & \text{otherwise.} \end{cases}$$

For $x \notin [0; 100]$ we assume that $\tau(x) = \infty$.

Note that $\lceil x \rceil$ is x rounded up to the nearest integer and $\lfloor x \rfloor$ is x rounded down to the nearest integer.

(a) Let the target function to be minimized be

$$\tau(x) = |x - 42|.$$

Let the initial state be $x_0 = 65$. **Execute bright side search (cf. Algorithm 1) in this setting for three time steps, i.e., compute x_1, x_2, x_3. Make sure to show all your computation steps. Briefly state why this run will never reach the global optimum at $x^* = 42$.** (8pts)

(b) We say an optimization algorithm converges on the global optimum if (given enough time) it is likely to come closer to the global optimum than it was initially and if, once it reaches the global optimum, it stays close to that point.
Give a target function $\tau^\dagger : \mathcal{X} \to \mathcal{T}$ in accordance with Algorithm 1 with one single global optimum so that bright side search always reaches the global optimum. Give that global optimum and briefly state why bright side search can reach it. Then show that bright side search still does not converge on that global optimum. (5pts)

(c) **State if bright side search is elitist. Briefly explain why.** (2pts)

(d) **State if bright side search is greedy. Briefly explain why.** (2pts)

5 Artificial Chemistries / Soups **16pts**

Recall the following definition from the lecture:

Definition 6 (artificial chemistry, soup)**.** Let $\mathcal{X}$ be a state space. Let $\mathcal{R}$ be a set of reaction rules $R \in \mathcal{R}$ where each R is a function $R : \mathcal{X}_R^{\text{dom}} \to \mathcal{X}_R^{\text{cod}}$ for $\mathcal{X}_R^{\text{dom}}, \mathcal{X}_R^{\text{cod}} \subseteq \wp^*(\mathcal{X})$. Let $\mathcal{A}(X, \mathcal{R})$ be the set of all *reaction materials* from $X \in \wp^*(\mathcal{X})$ for all rules in $\mathcal{R}$, i.e.,

$$\mathcal{A}(X, \mathcal{R}) = \left\{ (X', R) \;\middle|\; X' \subseteq X \;\wedge\; X' \in \mathcal{X}_R^{\text{dom}} \;\wedge\; R \in \mathcal{R} \right\}.$$

Let $A : \wp^*(\mathcal{X}) \to \wp^*(\mathcal{X}) \times \mathcal{R}$ be a possibly randomized or non-deterministic function that takes a multi-set $X \in \wp^*(\mathcal{X})$ and returns a multi-set of reactants and a reaction rule to perform so that $A(X) \in \mathcal{A}(X, \mathcal{R})$. The tuple $(\mathcal{X}, \mathcal{R}, A)$ defines an artificial chemistry. A multi-set $X \in \wp^*(\mathcal{X})$ of an artificial chemistry is also called soup; each element $x \in X$ is also called a particle. The evolution of a soup for g generations is a sequence $\langle X_0, .., X_g \rangle$ so that X_0 is given as the initial soup and, for all t with $0 \leq t \leq g - 1$

$$X_{t+1} = (X_t \setminus X') \uplus R(X')$$

where $(X', R) = A(X_t)$.
Reaction rules are usually written in the form $R = \sum_i x_i \rightharpoonup \sum_j y_j$ for $x_i, y_j \in \mathcal{X}$, $i, j \in \mathbb{N}$, $\{x_i\}_i \in \mathcal{X}_R^{\text{dom}}, \{y_j\}_j \in \mathcal{X}_R^{\text{cod}}$, and $R(\{x_i\}_i) = \{y_j\}_j$.

For the following tasks we define an artificial chemistry $\mathcal{C} = (\mathcal{X}, \mathcal{R}, A)$ where $\mathcal{X} = \{(\textsc{Snail}, \mathit{size}), (\textsc{Shell}, \mathit{size}), \textsc{Fish}, (\textsc{HermitCrab}, \mathit{size}), (\textsc{ProtectedHermitCrab}, \mathit{size})\}$ for all $\mathit{size} \in [1; 10] \subset \mathbb{N}$, $\mathcal{R} = \{R_1, .., R_6\}$ where

$$\begin{aligned}
R_1 &= (\textsc{Snail}, \mathit{size}) \rightharpoonup (\textsc{Snail}, \mathit{size}'), \\
R_2 &= (\textsc{Snail}, \mathit{size}) + \textsc{Fish} \rightharpoonup (\textsc{Shell}, \mathit{size}) + \textsc{Fish}, \\
R_3 &= (\textsc{HermitCrab}, \mathit{size}) \rightharpoonup (\textsc{HermitCrab}, \mathit{size}'), \\
R_4 &= (\textsc{HermitCrab}, \mathit{size}) + \textsc{Fish} \rightharpoonup \textsc{Fish}, \\
R_5 &= \textsc{Fish} + \textsc{Fish} \rightharpoonup \textsc{Fish} + \textsc{Fish} + \textsc{Fish}, \\
R_6 &= (\textsc{ProtectedHermitCrab}, \mathit{size}) + \textsc{Fish} + \textsc{Fish} + \textsc{Fish} \\
&\qquad \rightharpoonup (\textsc{Shell}, \mathit{size}) + \textsc{Fish} + \textsc{Fish} + \textsc{Fish},
\end{aligned}$$

with $\mathit{size}' = \min\{\mathit{size} + 1,\ 10\}$ for all $\mathit{size} \in [1; 10] \subset \mathbb{N}$, and $A(X) \sim \mathcal{A}(X, \mathcal{R})$.

(a) **Give a possible evolution of the soup**

$$X_0 = \{\textsc{Fish}, \textsc{Fish}, (\textsc{Snail}, 6), (\textsc{ProtectedHermitCrab}, 7)\}$$

for as many time steps as necessary until <u>two</u> $(\textsc{Shell}, 7)$ **appear.** (Hint: Instead of random choices, you can decide which rules to apply to make the process quicker. Use the table below.) (4pts)

time step	chosen rule	current soup
0	none	$\{\textsc{Fish}, \textsc{Fish}, (\textsc{Snail}, 6), (\textsc{ProtectedHermitCrab}, 7)\}$
1		
2		
3		
4		
5		

(b) Let $X^\dagger = \{(\textsc{Snail}, 1), (\textsc{HermitCrab}, 2), \textsc{Fish}\}$. **Give the full resulting value of** $\mathcal{A}(X^\dagger, \mathcal{R})$**.** (6pts)

(c) Let still $X^\dagger = \{(\textsc{Snail}, 1), (\textsc{HermitCrab}, 2), \textsc{Fish}\}$. **Give (in a mathematically concise way) all possible final soups (i.e., soups that do not change anymore through any reaction rules) if we keep evolving** $X^\dagger$ **forever.** (3pts)

(d) Our zoologist friends have observed that a HermitCrab, when it comes across a Shell of exactly equal size, can become a ProtectedHermitCrab of that size. **Give a reaction rule** R_7 **that can be added to the artificial chemistry** $\mathcal{C}$ **and describes that behavior.** (3pts)

6 Evolutionary Computing **17pts**

If necessary, you can recollect the definition for evolutionary algorithms as used in the lecture on this page. (Hint: Try the tasks first before reading this lengthy definition again.)

Algorithm 2 (typical evolutionary algorithm)**.** Let $\mathcal{E} = (\mathcal{X}, \mathcal{T}, \phi, E, \langle X_u \rangle_{0 \leq u \leq t})$ be a population-based optimization process.

- Let $\sigma_N^{survivors}, \sigma_N^{parents} : \wp^*(\mathcal{X}) \to \wp^*(\mathcal{X})$ be (possibly randomized or non-deterministic) selection functions that return $N \in \mathbb{N}$ individuals, i.e., $|\sigma_N(X)| = N$ and $\sigma_N(X) \subseteq X$ for all X. They may possibly depend on $\mathcal{E}$ (most commonly ϕ). Let $\varrho_N^{mutants} : \wp^*(\mathcal{X}) \to \wp^*(\mathcal{X})$ be a special selection function that, given a population X, returns a random subset $X' \subseteq X$ so that $|X'| = N$. Note that $\varrho_{|X|}(X) = X$ for all X.

- Let $mutate : \mathcal{X} \to \mathcal{X}$ be a (possibly randomized or non-deterministic) single-individual mutation function. We write $mutate[\_] : \wp^*(\mathcal{X}) \to \wp^*(\mathcal{X})$ for the per-individual application of $mutate$, i.e., $mutate[X] = \{mutate(x) \ : \ x \in X\}$.

- Let $recombine : \mathcal{X} \times \mathcal{X} \to \mathcal{X}$ be a (possibly randomized or non-deterministic) two-parent recombination function. We write $recombine[\_] : \wp^*(\mathcal{X}) \to \wp^*(\mathcal{X})$ for the random-pairing application of $recombine$, i.e., $recombine[X] = \{recombine(x_1, x_2)\} \uplus recombine[X \setminus \{x_1, x_2\}]$ with $x_1, x_2 \sim X$ and $recombine[\{x\}] = \emptyset$ for all x as well as $recombine[\emptyset] = \emptyset$.

- Let $migrate : \emptyset \to \mathcal{X}$ be a (possibly randomized or non-deterministic) individual creation function. We write $migrate_N : \emptyset \to \wp^*(\mathcal{X})$ with $N \in \mathbb{N}$ for the iterated application of $migrate$, i.e., $migrate_N[\,] = \{migrate()\} \uplus migrate_{N-1}[\,]$ with $migrate_0[\,] = \emptyset$.

The process $\mathcal{E}$ continues via a typical evolutionary algorithm if E has the form

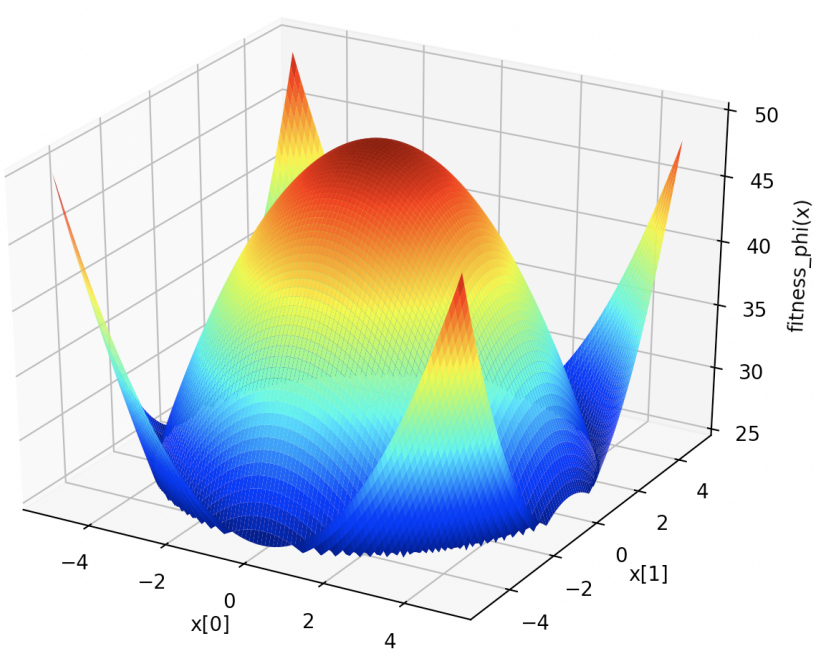
$$\begin{aligned} E(\langle X_u \rangle_{0 \leq u \leq t}, \phi) = X_{t+1} = \sigma_{|X_t|}^{survivors}\big(X_t \\ \uplus\ & mutate[\varrho_M^{mutants}(X_t)] \\ \uplus\ & recombine[\sigma_{2C}^{parents}(X_t)] \\ \uplus\ & migrate_H[\,]\big) \end{aligned}$$

where $M \in \mathbb{N}$ is the number of mutants produced per generation, $C \in \mathbb{N}$ is the number of children produced per generation, and $H \in \mathbb{N}$ is the number of migrants (sometimes called hypermutants) generated per generation. Especially M is often expressed as a rate for random choice within the population, i.e., $M = \lceil m \cdot |X_t| \rceil$ where $m \in \mathbb{R}$ is called mutation rate.

We now consider an evolutionary algorithm that searches through individuals within $\mathcal{X} = Q^2$ where $Q = [-5; +5] \subset \mathbb{R}$. It does so by minimizing a fitness function $\phi : Q^2 \to \mathbb{R}$, which is given via the following Python function:

```
def fitness_phi(x):
    return max(x[0]**2 + x[1]**2, 50 - x[0]**2 - x[1]**2)
```

The resulting solution landscape can be seen as shown below:



(a) This fitness function features multiple global optima. **State two different global optima that minimize** `fitness_phi`. (2pts.)

(b) **Shortly describe a technique that should help us find multiple different global optima within one single run of the evolutionary algorithm.** (3pts)

(c) Consider the following two recombination functions given in Python. Note that the Python function `random.random()` returns a random floating point number n between `0.0` (inclusive) and `1.0` (exclusive), i.e., $n \sim [0.0; 1.0)$.

```
import random

def recombine_avg(parent_a, parent_b):
    child = [0.0, 0.0]
    child[0] = (parent_a[0] + parent_b[0]) / 2.0
    child[1] = (parent_a[1] + parent_b[1]) / 2.0
    return child

def recombine_xover(parent_a, parent_b):
    if random.random() < 0.5:
        return [parent_a[0], parent_b[1]]
    else:
        return [parent_b[0], parent_a[1]]
```

Which of the two recombination functions `recombine_avg` and `recombine_xover` is better suited for the optimization problem given by `fitness_phi`? Explain why. (3pts)

(d) We define a *nice* mutation function to have the following qualities:

- It does not return solution candidates that lie out of the search space's boundaries.
- Its effects are random and not affected by fitness.
- Its effects work the same way in all the individual's dimensions.
- Its effects *completely* change the original individual *only very rarely at most*.
- There is *no point* within the search space that can *never* be reached via mutation, provided any arbitrary individual as a starting point and infinite time.

In a suitable formalism (Python, pseudo-code, mathematics, ...) give a *nice* mutation function for the above evolutionary algorithm. (5pts)

(e) Consider that we now construct an evolutionary algorithm with local search, i.e., we augment the evolutionary algorithm by using a greedy gradient-based search algorithm on each single individual before the fitness evaluation. **Given the fitness function `fitness_phi` (and thus the solution landscape above), what should we expect to happen when running this augmented evolutionary algorithm? As precisely as possible, explain why this setup is reasonable (or not) to optimize for the given fitness function `fitness_phi`?** (4pts)